



UNIVERSIDAD PERUANA DE CIENCIAS APLICADAS

FACULTAD DE INGENIERÍA

PROGRAMA ACADÉMICO DE CIENCIAS DE LA COMPUTACIÓN

CURSO: 1ACC0065 - PROGRAMACIÓN CONCURRENTES

NRC: 8646

PC1

INTEGRANTES:

Manchay Paredes, Lucero Salome (U202216120)

Mayhua Hinostroza, José Antonio (U202218044)

Peña Cardenas, Jhamil Brijan (U2017114492)

202620

ÍNDICE

1. Resumen del trabajo.....	3
2. Objetivos del trabajo.....	3
2.1 Objetivo general.....	3
2.2 Objetivos específicos.....	3
3. Explicación de los temas investigados.....	3
3.1 Concurrencia en Go aplicada a Machine Learning.....	4
3.2 Regresión Lineal concurrente.....	4
3.3 Cadena de búsqueda utilizada.....	4
3.4 Tabla de papers por integrante.....	5
Paper 1 (Integrante 1 - José Antonio Mayhua Hinostroza).....	5
Paper 2 (Integrante 2 - Lucero Salome Manchay Paredes).....	7
Paper 3 (Integrante 3 - Jhamil Brijan Peña Cardenas).....	8
Elección Final:.....	11
4. Análisis del caso de uso.....	11
4.1 Elección y sustentación.....	11
4.2 Selección del dataset.....	12
5. Procedimiento de limpieza del dataset.....	13
5.1 Pasos de depuración aplicados.....	13
6. Trabajo en GitHub.....	14
7. Referencias bibliográficas (formato APA).....	14

1. Resumen del trabajo

El presente trabajo aborda la optimización de costos y recursos hospitalarios mediante el uso de modelos de machine learning implementados con mecanismos de concurrencia en el lenguaje Go. El caso de uso se enmarca en el Objetivo de Desarrollo Sostenible 3 (Salud y bienestar) y busca analizar los costos médicos y los días de estancia hospitalaria para apoyar la asignación eficiente de camas y presupuestos en centros de salud.

Se plantea el uso del modelo de Regresión Lineal seleccionado por el grupo, para predecir el costo total de atención (Total Costs) en función de los días de estancia (Length of Stay) y variables categóricas asociadas al diagnóstico y tipo de pago. El modelo será implementado de forma secuencial y concurrente (goroutines, sync.Mutex, sync.WaitGroup, patrones Worker Pool / Pipeline) sobre un dataset de más de un millón de registros, con el fin de medir el speedup obtenido y evaluar la escalabilidad de la solución.

Este entregable (PC1) documenta la investigación bibliográfica inicial, la sustentación del caso de uso, la selección del dataset y el procedimiento de limpieza de datos que servirá de base para los entregables posteriores (PC2 y TP).

2. Objetivos del trabajo

2.1 Objetivo general

Diseñar e implementar un algoritmo concurrente en Go que aplique el modelo de Regresión Lineal sobre un dataset hospitalario de más de un millón de registros, optimizando el tiempo de ejecución y evaluando su escalabilidad frente a una implementación secuencial equivalente.

2.2 Objetivos específicos

- Investigar el estado del arte sobre el uso de concurrencia en algoritmos de machine learning, identificando beneficios y limitaciones reportados en la literatura académica reciente.
- Seleccionar y depurar un dataset público de salud con más de un millón de registros, adecuado para el entrenamiento del modelo propuesto.
- Sustentar la elección del caso de uso y su vínculo con el ODS 3 (Salud y bienestar).
- Modelar la lógica de sincronización en Promela y verificarla formalmente en Spin (entregables posteriores).
- Implementar y comparar versiones secuenciales y concurrentes del modelo, midiendo el speedup y el uso de recursos de cómputo (entregables posteriores).

3. Explicación de los temas investigados

El grupo investiga dos ejes temáticos relacionados con el caso de uso: (1) programación concurrente en Go aplicada a machine learning, y (2) el modelo de Regresión Lineal y sus variantes de paralelización. A continuación, cada integrante documenta 1 paper académico de

los últimos cinco años, para un total de 3 artículos, siguiendo el formato exigido por la rúbrica.

3.1 Concurrencia en Go aplicada a Machine Learning

Go ofrece concurrencia nativa mediante goroutines (hilos ligeros gestionados por el runtime) y canales, junto con primitivas de sincronización como `sync.Mutex` (exclusión mutua) y `sync.WaitGroup` (espera de finalización de tareas). Estos mecanismos permiten implementar de forma concurrente el entrenamiento o la inferencia de modelos de ML sin depender de librerías externas, distribuyendo el dataset entre varios workers y agregando resultados parciales, típicamente mediante patrones Worker Pool o Pipeline.

3.2 Regresión Lineal concurrente

La Regresión Lineal, al basarse en operaciones matriciales y sumatorias independientes por bloque de datos, es un candidato natural para implementación concurrente mediante particionamiento de datos (data partitioning): cada goroutine calcula sumas parciales de forma concurrente e independiente (por ejemplo, para el método de mínimos cuadrados o el descenso de gradiente por lotes), que luego se combinan de forma sincronizada usando primitivas como `sync.Mutex` y `sync.WaitGroup`.

La concurrencia permite que múltiples goroutines trabajen sobre particiones del dataset de forma intercalada, mejorando la escalabilidad y el tiempo de ejecución en comparación con una implementación secuencial pura. En hardware multi-núcleo, esta concurrencia se traduce en paralelismo real, logrando un speedup medible.

3.3 Cadena de búsqueda utilizada

Como cadena base para orientar la búsqueda bibliográfica se consideró la combinación **"Machine Learning" AND "concurrent programming"**. A partir de estos conceptos, cada integrante refinó la búsqueda según el enfoque específico de su artículo, incorporando términos relacionados con Regresión Lineal, aprendizaje distribuido, procesamiento paralelo, GPU, Stochastic Gradient Descent y mecanismos de sincronización.

Como punto de partida se consideraron los conceptos Machine Learning, concurrencia y Regresión Lineal. Posteriormente, cada integrante adaptó la cadena de búsqueda a las características específicas de su artículo, incorporando términos asociados a aprendizaje distribuido, procesamiento paralelo, GPU, gradient descent y sincronización.

Ejemplo de refinamiento por integrante:

"Machine learning" AND "linear regression" AND "concurrent programming"

Integrante 1: José Antonio Mayhua Hinostroza

- **Paper:** Linear Regression With Distributed Learning: A Generalization Error Perspective
- **Cadena de búsqueda:** "machine learning" AND "linear regression" AND "distributed learning"

Integrante 2: Lucero Salome Manchay Paredes:

- **Paper:** Recursive SQL and GPU-support for in-database machine learning
- **Cadena de búsqueda:** "machine learning" AND "linear regression" AND ("parallel" OR "GPU") AND "gradient descent"

Integrante 3: Jhamil Brijan Peña Cardenas,

- **Paper:** Consistent Lock-free Parallel Stochastic Gradient Descent for Fast and Stable Convergence
- **Cadena de búsqueda:** "machine learning" AND "stochastic gradient descent" AND ("parallel" OR "lock-free")

3.4 Tabla de papers por integrante

Cada integrante debe completar una fila por paper (1 paper por integrante, 3 en total), usando fuentes indexadas (Scopus, IEEE Xplore, ACM Digital Library) de los últimos 5 años.

Paper 1 (Integrante 1 - José Antonio Mayhua Hinostroza)

Campo	Contenido
Título del paper	Linear Regression with Distributed Learning: A Generalization Error Perspective
Autores	Martin Hellkvist, Ayça Özçelikkale y Anders Ahlén
Año de publicación	2021
Motivación	El aprendizaje distribuido permite abordar tareas de aprendizaje a gran escala repartiendo la carga computacional entre múltiples nodos de una red. Sin embargo, los autores señalan que gran parte de los estudios existentes se concentra en el rendimiento sobre los datos de entrenamiento y no analiza suficientemente cómo la distribución del modelo afecta su capacidad para generalizar sobre datos no observados. Por ello, el trabajo estudia la Regresión Lineal distribuida desde la perspectiva del error de generalización, buscando identificar posibles problemas derivados del particionamiento del modelo.
Problema que resuelve	El trabajo aborda la falta de conocimiento sobre cómo el particionamiento de los parámetros de un modelo de Regresión Lineal entre diferentes nodos afecta el error de generalización. En particular, busca determinar si una solución distribuida que obtiene

	un bajo error de entrenamiento conserva necesariamente un buen rendimiento sobre datos nuevos. Los autores muestran que esto no siempre ocurre: el error de generalización puede ser bastante mayor que el de una solución centralizada aun cuando ambas tengan errores de entrenamiento similares.
Propuesta de solución	Los autores estudian una Regresión Lineal en la cual los parámetros desconocidos del modelo se dividen entre K nodos. Para obtener la solución distribuida usan el algoritmo COCOA, que permite que cada nodo resuelva un subproblema local y comunique su contribución a un nodo central. Después analizan de forma matemática el error de generalización producido por distintas formas de particionar los parámetros y establecen cotas probabilísticas para datos gaussianos isotrópicos, gaussianos correlacionados y subgaussianos. La propuesta se complementa con experimentos sobre datos sintéticos y datos reales de MNIST.
Algoritmos usados / pseudocódigo	Se usa Regresión Lineal regularizada mediante mínimos cuadrados y el algoritmo/framework de optimización distribuida COCOA. La matriz de regresores se divide por columnas entre K nodos. En cada iteración, cada nodo calcula una actualización de sus parámetros locales mediante un problema de optimización convexa; después envía su contribución a un nodo central, que calcula una variable agregada y la redistribuye a los nodos para iniciar la siguiente iteración.
Servicios, frameworks, bibliotecas, lenguaje, patrones	El artículo no especifica un lenguaje de programación, biblioteca o framework de software concreto como Python, TensorFlow, PyTorch, Go o MATLAB. Su contribución es principalmente matemática y algorítmica. Usa el algoritmo COCOA como método de optimización distribuida y una arquitectura de múltiples nodos con coordinación y agregación central. Para los experimentos usa datos sintéticos y el dataset MNIST. Por ello, no sería correcto atribuirle herramientas de programación que el artículo no declara.
Opinión crítica del estudiante	Considero que el artículo muestra que distribuir un modelo de Machine Learning puede ser beneficioso para repartir la carga computacional de problemas de gran escala, pero también deja claro que la distribución debe diseñarse con cuidado. Un aspecto relevante es que un bajo error de entrenamiento no garantiza un bajo error de generalización: dependiendo de cómo se dividan los parámetros entre los nodos, el rendimiento sobre datos nuevos puede empeorar bastante. Esto es importante para nuestro proyecto porque muestra que, al aplicar concurrencia o paralelismo, no basta con aumentar el número de workers para obtener una mejor

	solución; además del tiempo de ejecución y el speedup, es necesario analizar cómo se divide el trabajo y verificar que la calidad del modelo no se vea afectada.
Link de acceso al paper	DOI: https://ieeexplore.ieee.org/document/9520293 Acceso abierto al paper: https://www.researchgate.net/publication/354044522_Linear_Regression_With_Distributed_Learning_A_Generalization_Error_Perspective

Paper 2 (Integrante 2 - Lucero Salome Manchay Paredes)

Campo	Contenido
Título del paper	Recursive SQL and GPU-support for in-database machine learning
Autores	Maximilian E. Schüle, Harald Lang, Maximilian Springer, Alfons Kemper, Thomas Neumann y Stephan Günnemann.
Año de publicación	2022
Motivación	Los autores identifican que los pipelines tradicionales de Machine Learning suelen extraer los datos de una base de datos hacia herramientas externas como NumPy, TensorFlow o PyTorch para realizar el entrenamiento. Esta extracción puede resultar costosa cuando se trabaja con grandes volúmenes de información. Por ello, investigan la posibilidad de ejecutar directamente dentro del sistema de base de datos las etapas de preprocesamiento, entrenamiento y validación, aprovechando además procesamiento paralelo en CPU y GPU.
Problema que resuelve	Busca reducir el costo de transferencia y procesamiento requerido para entrenar modelos de Machine Learning sobre grandes conjuntos de datos. Además, estudia cómo paralelizar el entrenamiento mediante múltiples threads, workers y GPUs sin que los mecanismos de sincronización reduzcan excesivamente el rendimiento. El artículo utiliza explícitamente Regresión Lineal como uno de sus modelos de evaluación.
Propuesta de solución	Los autores extienden el sistema de base de datos Umbra para expresar un pipeline completo de Machine Learning mediante SQL. Incorporan diferenciación automática y un operador de Gradient Descent capaz de generar código LLVM y ejecutar el entrenamiento sobre GPU. Para Regresión Lineal, los datos se procesan por mini-batches y se distribuyen entre threads/workers. Se comparan estrategias con sincronización global, actualizaciones sin bloqueo y modelos locales.

Algoritmos usados / pseudocódigo	Se utiliza Regresión Lineal, Mini-batch Gradient Descent, diferenciación automática y técnicas paralelas de reducción/agregación de gradientes. El artículo presenta distintas estrategias de sincronización: workers sincronizados, workers con actualizaciones globales y workers con modelos locales. Cada worker procesa una parte de los datos, calcula su gradiente y posteriormente los resultados son agregados para actualizar los pesos del modelo.
Servicios, frameworks, bibliotecas, lenguaje, patrones	Umbra como DBMS experimental; SQL para representar el pipeline; LLVM para generación de código; CUDA para ejecución en GPU y cuBLAS para operaciones vectoriales/matriciales. Se utilizan CPU threads, GPU threads, múltiples workers, particionamiento de datos, mini-batches, reducción de gradientes y modelos globales/locales.
Opinión crítica del estudiante	Considero que el trabajo demuestra de forma clara el beneficio de paralelizar el entrenamiento de Regresión Lineal, especialmente cuando se procesan grandes cantidades de datos. El uso de varios workers permite incrementar el throughput y reducir el tiempo de ejecución; sin embargo, el artículo también evidencia que la concurrencia no garantiza automáticamente un mayor rendimiento. Cuando se utilizan locks o sincronización bloqueante, algunos workers permanecen inactivos esperando a otros, reduciendo el speedup obtenido. Esto muestra que el diseño del mecanismo de sincronización es tan importante como el número de workers utilizados. Esta observación es especialmente relevante para nuestra implementación en Go, donde será necesario controlar correctamente el uso de goroutines, sync.Mutex y sync.WaitGroup.
Link de acceso al paper	https://link.springer.com/article/10.1007/s10619-022-07417-7

Paper 3 (Integrante 3 - Jhamil Brijan Peña Cardenas)

Campo	Contenido
Título del paper	Consistent Lock-free Parallel Stochastic Gradient Descent for Fast and Stable Convergence
Autores	Karl Bäckström, Ivan Walulya, Marina Papatriantafilou y Philippos Tsigas (Chalmers University of Technology, Suecia).
Año de publicación	2021 (35th IEEE International Parallel & Distributed Processing Symposium, IPDPS 2021)

Motivación		El Descenso de Gradiente Estocástico (SGD) es el motor de optimización de numerosos algoritmos de Machine Learning, incluida la Regresión Lineal. Los autores observan que, si bien existen variantes asíncronas de SGD sobre memoria compartida (AsyncSGD y HOGWILD!) que reducen el costo de coordinación frente a la parallelización síncrona, la literatura no explica con claridad en qué situaciones prácticas resultan realmente útiles ni qué papel cumplen los mecanismos de sincronización y de consistencia. Señalan además que muchos trabajos previos miden la convergencia en número de iteraciones (eficiencia estadística) y no en tiempo real de reloj, lo que puede llevar a conclusiones engañosas sobre el beneficio verdadero de la concurrencia.
Problema que resuelve	que	El artículo aborda el compromiso entre consistencia y sobrecarga de sincronización en SGD concurrente sobre memoria compartida. Cuando varios hilos leen y actualizan simultáneamente el vector de parámetros, aparecen dos fenómenos que degradan el aprendizaje: la obsolescencia (staleness), es decir, aplicar un gradiente calculado sobre una versión ya desactualizada de los parámetros, y la inconsistencia, que surge cuando las lecturas y escrituras no coordinadas dejan el vector en un estado intermedio. Usar bloqueos (mutex) elimina la inconsistencia pero introduce contención y espera; eliminarlos por completo (estilo HOGWILD!) elimina la espera pero rompe la consistencia. El trabajo busca determinar si vale la pena pagar el costo de garantizar consistencia y cómo hacerlo sin bloqueos.
Propuesta de solución	de	Los autores proponen Leashed-SGD, un marco algorítmico extensible de implementaciones de AsyncSGD que preservan la consistencia empleando sincronización lock-free (libre de bloqueos) en lugar de exclusión mutua. El componente central es la estructura de datos ParameterVector, que encapsula el vector de parámetros junto con metadatos de control (número de secuencia, contador de lectores activos y banderas de obsolescencia y borrado) y permite reciclar memoria de forma segura. Cada hilo calcula su gradiente localmente y publica el resultado mediante una única operación atómica Compare-And-Swap (CAS) sobre un puntero global. Además incorporan un umbral de persistencia (T_p) que limita el número de reintentos de CAS y actúa como mecanismo autorregulador de la contención.
Algoritmos usados / pseudocódigo		Se emplean SGD secuencial, AsyncSGD basado en bloqueos (mutex), HOGWILD! (sin sincronización) y el propuesto Leashed-SGD. El artículo presenta pseudocódigo para los cuatro. En AsyncSGD cada hilo bloquea el mutex, copia los parámetros,

	libera, calcula el gradiente, y vuelve a bloquear para aplicar la actualización. En HOGWILD! se eliminan las llamadas de bloqueo. En Leashed-SGD (Algoritmo 3) el hilo obtiene un puntero al vector más reciente mediante la función <code>latest_pointer()</code> dentro de un bucle de reintento, calcula el gradiente sin copiar, crea una nueva instancia de <code>ParameterVector</code> con la actualización aplicada e intenta publicarla con CAS en el bucle LAU-SPC (<code>LoadAndUpdate-StorePersistenceConditional</code>); si el CAS falla, otro hilo se adelantó y se reintenta hasta superar T_p. Se demuestra formalmente (Lema 1) que las lecturas y actualizaciones cumplen lock-freedom y atomicidad, y (Lema 2) que el reciclaje de memoria es seguro y acotado.
Servicios, frameworks, bibliotecas, lenguaje, patrones	El marco está implementado en C++, usando OpenMP para el cómputo paralelo en memoria compartida y la biblioteca Eigen para las operaciones numéricas, extendiendo la biblioteca de deep learning MiniDNN. Los patrones y mecanismos empleados son: sincronización lock-free mediante operaciones atómicas read-modify-write (CAS y Fetch-And-Add), exclusión mutua con locks como línea base de comparación, bucles de reintento, conteo atómico de lectores, reciclaje dinámico de memoria y particionamiento del trabajo entre hilos. Las pruebas se ejecutaron sobre un Intel Xeon E5-2695 de 36 núcleos con hyper-threading, entrenando MLP y CNN sobre el dataset MNIST con hasta 68 hilos concurrentes.
Opinión crítica del estudiante	Considero que este artículo aporta la lección más directamente aplicable a nuestro trabajo: la concurrencia no mejora el rendimiento por sí sola, sino que su beneficio depende críticamente del mecanismo de sincronización elegido. Los autores muestran que la alternativa aparentemente más rápida, eliminar toda sincronización, como HOGWILD!, se vuelve inestable al aumentar el paralelismo, al punto de que con 68 hilos ninguna ejecución de las líneas base logró converger, mientras que la variante lock-free consistente sí lo hizo y alcanzó mejoras de entre 20-80% y hasta 4x en tiempo real de convergencia, además de reducir el uso de memoria. Esto evidencia un perjuicio importante de la programación concurrente mal diseñada: la contención y la obsolescencia pueden anular por completo la ganancia teórica del speedup. Un aspecto valioso desde el punto de vista metodológico es su insistencia en medir el tiempo real de reloj y no solo el número de iteraciones, criterio que adoptaremos al comparar nuestras versiones secuencial y concurrente en Go. Como limitación, la evaluación se centra en redes neuronales (MLP y

	CNN) y no en Regresión Lineal, aunque los propios autores señalan que SGD es la base de modelos convexos como la regresión lineal y que el marco es aplicable a cualquier problema de optimización. Asimismo, el uso de C++ con OpenMP difiere de nuestro entorno en Go, aunque los conceptos de exclusión mutua y contención se trasladan directamente al uso de sync.Mutex y sync.WaitGroup.
Link de acceso al paper	DOI / IEEE Xplore: https://ieeexplore.ieee.org/document/9460457 Acceso abierto al paper: https://arxiv.org/abs/2102.09032

Elección Final:

Tras revisar los tres artículos, el grupo selecciona como referencia principal el trabajo de Schüle et al. (2022), Recursive SQL and GPU-support for in-database machine learning, debido a que presenta el mayor grado de correspondencia con el caso de uso propuesto. El artículo emplea explícitamente Regresión Lineal y Gradient Descent, distribuyendo el procesamiento de los datos entre múltiples workers y evaluando diferentes estrategias de sincronización y actualización del modelo.

Esta propuesta resulta especialmente relevante para el proyecto debido a que la solución planteada también utilizará Regresión Lineal sobre un dataset de gran tamaño y posteriormente estudiará su procesamiento mediante mecanismos de concurrencia en Go. Aunque las tecnologías empleadas en el artículo, como CPU threads, CUDA y GPU, difieren del entorno del proyecto, los principios de particionamiento de datos, cálculo independiente de gradientes y agregación de resultados pueden trasladarse conceptualmente al uso de goroutines.

El trabajo de Bäckström et al. (2021) se adopta como referencia complementaria para el análisis de sincronización, ya que demuestra que incrementar el paralelismo no garantiza por sí mismo una mejora del rendimiento y que la contención, la consistencia y el mecanismo de actualización de los parámetros influyen directamente en el tiempo de ejecución.

Finalmente, Hellkvist et al. (2021) aporta una perspectiva complementaria sobre el particionamiento en Regresión Lineal distribuida y advierte que una distribución inadecuada del problema puede afectar la capacidad de generalización del modelo.

4. Análisis del caso de uso

4.1 Elección y sustentación

El caso de uso seleccionado consiste en el análisis de costos médicos y días de estancia hospitalaria para apoyar la optimización de la asignación de camas y presupuestos en centros de salud, en línea con el ODS 3 (Salud y bienestar). Se eligió este caso por tres razones principales:

- Relevancia social: la gestión eficiente de recursos hospitalarios impacta directamente en la calidad y oportunidad de la atención médica, especialmente en sistemas de salud con recursos limitados.
- Naturaleza tabular de los datos: el análisis combina variables numéricas, como Length of Stay, APR Severity of Illness Code y Total Costs, con variables categóricas como Age Group, Type of Admission, APR MDC Description, APR Medical Surgical Description, Payment Typology 1 y Emergency Department Indicator. Esto permite trabajar con información estructurada sin requerir procesamiento de imágenes médicas ni texto clínico libre.
- Escalabilidad del problema: los sistemas hospitalarios generan volúmenes del orden de millones de registros de atención y altas hospitalarias, lo que proporciona un escenario adecuado para estudiar posteriormente técnicas de procesamiento concurrente.

El modelo elegido, Regresión Lineal, permitirá estimar el costo total de atención (Total Costs) a partir de la duración de la hospitalización (Length of Stay) y variables clínicas y administrativas asociadas al episodio hospitalario, como severidad, categoría diagnóstica principal, tipo de admisión, grupo etario y tipo de pago. Esta estimación puede contribuir a la planificación presupuestaria y al análisis del consumo de recursos hospitalarios.

4.2 Selección del dataset

Se evaluó inicialmente el dataset público "Medicare Inpatient Prospective Payment System - Provider Summary" (CMS), que contiene variables agregadas por hospital y DRG (total_discharges, average_covered_charges, average_total_payments). Sin embargo, este dataset solo contiene aproximadamente 163,000 registros (cerca de 3,000 hospitales por ~100 categorías de procedimiento), por lo que no cumple el requisito de más de un millón de registros exigido por el curso.

Se evaluó también el CMS Data Entrepreneurs' Synthetic Public Use File (DE-SynPUF), que sí supera holgadamente el millón de registros, pero corresponde a reclamos sintéticos de 2008-2010, fuera de la ventana de vigencia de 5 años que el grupo decidió aplicar de forma consistente tanto a los papers académicos como al dataset.

Por ello, el grupo selecciona como dataset definitivo el Hospital Inpatient Discharges (SPARCS De-Identified) del año 2022, publicado por el New York State Department of Health (2024) a través del portal público Health Data NY. El SPARCS (Statewide Planning and Research Cooperative System) es un sistema de reporte obligatorio que recopila el detalle de cada alta hospitalaria en el estado de Nueva York, y su versión desidentificada es de acceso público y gratuito.

El dataset SPARCS 2022 contiene 2,103,433 registros a nivel de alta hospitalaria individual. Para el caso de uso se seleccionaron variables relacionadas con duración, características clínicas y administrativas de la hospitalización: Length of Stay, Age Group, Type of Admission, APR Severity of Illness Code, APR MDC Description, APR Medical Surgical Description, Payment Typology 1, Emergency Department Indicator y Total Costs. Esta

última constituye la variable objetivo del modelo de Regresión Lineal (New York State Department of Health, 2024).

Como respaldo académico de la elección, Jain et al. (2024) utilizaron una fuente del sistema SPARCS con aproximadamente 2.3 millones de registros desidentificados de altas hospitalarias para desarrollar modelos de Machine Learning orientados a la predicción de la duración de la estancia hospitalaria, lo que respalda la pertinencia de esta fuente para el caso de uso planteado.

5. Procedimiento de limpieza del dataset

El procedimiento de depuración se implementó en Go sobre el dataset Hospital Inpatient Discharges (SPARCS De-Identified): 2022. Las reglas aplicadas fueron definidas a partir del análisis de las características y calidad de los datos, con el objetivo de conservar únicamente la información necesaria para el posterior modelo de Regresión Lineal y evitar la eliminación injustificada de hospitalizaciones válidas.

5.1 Pasos de depuración aplicados

- **Carga y lectura:** se procesó el archivo CSV completo correspondiente a SPARCS 2022, compuesto por 2,103,433 registros de altas hospitalarias.
- **Selección de variables:** se conservaron Age Group, Length of Stay, Type of Admission, APR Severity of Illness Code, APR MDC Description, APR Medical Surgical Description, Payment Typology 1, Emergency Department Indicator y Total Costs. Asimismo, se generaron las variables auxiliares LOS_120_plus y Severity_unknown.
- **Conversión de Total Costs:** los separadores de miles y símbolos monetarios fueron eliminados y el valor se convirtió en un dato numérico. Los registros cuyo costo no podía convertirse correctamente o cuyo valor era menor o igual a cero fueron descartados.
- **Tratamiento de Length of Stay:** la estancia hospitalaria fue convertida a un valor entero. Los registros no convertibles o con una estancia menor o igual a cero fueron considerados inválidos.
- **Tratamiento de "120 +":** debido a que esta categoría representa hospitalizaciones de 120 días o más, se almacenó Length of Stay = 120 y se creó la variable binaria LOS_120_plus = 1, conservando así la información de censura presente en el dato original.
- **Tratamiento de severidad:** los valores válidos de APR Severity of Illness Code se conservaron en el rango 1-4. Los códigos 0, vacíos o inválidos se representaron mediante APR Severity of Illness Code = 0 y Severity_unknown = 1, evitando eliminar esos registros.
- **Variables categóricas:** se eliminaron espacios innecesarios y los valores vacíos se representaron mediante la categoría Unknown.
- **Valores atípicos:** durante el análisis se identificaron valores extremos en costos y días de estancia. Sin embargo, estos no fueron eliminados automáticamente mediante IQR,

debido a que una hospitalización prolongada o de alto costo puede representar un caso clínico válido y no necesariamente un error de calidad.

- **Resultado:** de los 2,103,433 registros procesados, se conservaron 2,103,432 y se descartó únicamente un registro por no cumplir la regla de validez establecida para Total Costs. Asimismo, se identificaron y transformaron 2,103 registros con Length of Stay = "120 +" y 636 registros con severidad desconocida. El resultado fue almacenado en SPARCS_2022_clean_go.csv.

Evidencia de ejecución:

```
PS C:\Users\SeuNg720\Documents\Grupo5-ProgramacionConcurrente> go run main.go

SPARCS 2022 - RESULTADO DE LIMPIEZA PC1
Registros leídos:                2103433
Registros conservados:           2103432
Registros descartados:           1
- Total Costs inválido/no > 0:   1
- Length of Stay inválido/no >0: 0
Casos Length of Stay = 120+:     2103
Casos de severidad desconocida:  636

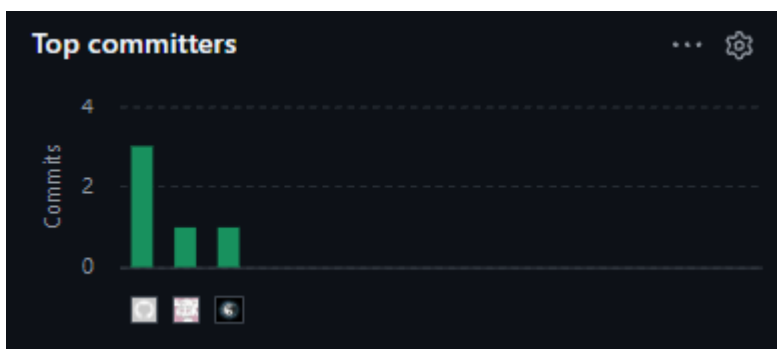
Dataset limpio generado en:
dataset/SPARCS 2022 clean go.csv
```

6. Trabajo en GitHub

El proyecto se aloja en un repositorio público de GitHub siguiendo el flujo Git Flow (ramas main/master, develop, feature/*, release/*, hotfix/*). Cada integrante realiza commits desde su propia cuenta para evidenciar su participación individual, conforme a lo exigido por la rúbrica.

Link del repositorio: <https://github.com/YuhiTTo/Grupo5-ProgramacionConcurrente.git>

Evidencia de commits:



7. Referencias bibliográficas (formato APA)

Hellkvist, M., Özçelikkale, A., & Ahlén, A. (2021). Linear regression with distributed learning: A generalization error perspective. *IEEE Transactions on Signal Processing*, 69, 5479–5495. <https://ieeexplore.ieee.org/document/9520293>

Schüle, M. E., Lang, H., Springer, M., Kemper, A., Neumann, T., & Günnemann, S. (2022). Recursive SQL and GPU-support for in-database machine learning. *Distributed and Parallel Databases*, 40(2–3), 205–259. <https://link.springer.com/article/10.1007/s10619-022-07417-7>

Bäckström, K., Walulya, I., Papatriantafilou, M., & Tsigas, P. (2021). Consistent lock-free parallel stochastic gradient descent for fast and stable convergence. En 2021 IEEE International Parallel and Distributed Processing Symposium (IPDPS) (pp. 423–432). IEEE. <https://ieeexplore.ieee.org/document/9460457>

Jain, R., Singh, M., Rao, A. R., & Garg, R. (2024). Predicting hospital length of stay using machine learning on a large open health dataset. *BMC Health Services Research*, 24, 860. <https://link.springer.com/article/10.1186/s12913-024-11238-y>

New York State Department of Health. (2024). Hospital inpatient discharges (SPARCS De-Identified): 2022 [Data set]. Health Data NY. https://health.data.ny.gov/Health/Hospital-Inpatient-Discharges-SPARCS-De-Identified/5dtw-tffi/about_data